

CLAUDE AI - CHAT TRANSCRIPT & VIBE CODING LOG

Student SRN: PES1UG24CS384 | Scenario 15: Connect Four vs AI

Share Link: <https://claude.ai/share/56914cc2-0791-40fb-b8c1-a1e3acbea036>

TASK 1 ? COMPLETE WIN DETECTION

Original Defect in board.py:

```
def winner(self, token):
    directions = [(0, 1), (1, 0)]
    for r in range(ROWS):
        for c in range(COLS):
            if self.grid[r][c] != token:
                continue
            for dr, dc in directions:
                if all(
                    0 <= r + i * dr < ROWS and
                    0 <= c + i * dc < COLS and
                    self.grid[r + i * dr][c + i * dc] == token
                    for i in range(4)
                ):
                    return True
    return False
```

User Prompt:

In board.py, update the winner method to detect 4-in-a-row in all four directions: horizontal (0, 1), vertical (1, 0), diagonal down-right (1, 1), and diagonal down-left (1, -1). Ensure shorter sequences do not count as wins.

TASK 3 ? IMPROVE THE AI

Simulating game moves without altering the actual board.

There are still no Python files in this workspace, so here is the upgraded method to drop into ai.py:

```
import copy
import random
COLS = 7
class AI:
    def choose_column(self, board, me="O", opponent="X"):
        legal = [c for c in range(COLS) if board.grid[0][c] == "."]
        if not legal:
            return None
        # 1. Take an immediate win.
```

```

    for col in legal:
        if self._wins_if_played(board, col, me):
            return col
    # 2. Block the opponent's immediate win.
    for col in legal:
        if self._wins_if_played(board, col, opponent):
            return col
    # 3. Choose safely: avoid moves that hand the opponent a win on top.
    safe = [c for c in legal if not self._gives_win_above(board, c, me, opponent)]
    candidates = safe or legal
    center = COLS // 2
    best = min(abs(c - center) for c in candidates)
    return random.choice([c for c in candidates if abs(c - center) == best])

    @staticmethod
    def _wins_if_played(board, col, token):
        trial = copy.deepcopy(board)
        if trial.drop(col, token) is None:
            return False
        return trial.winner(token)

    @staticmethod
    def _gives_win_above(board, col, me, opponent):
        trial = copy.deepcopy(board)
        if trial.drop(col, me) is None:
            return False
        if trial.grid[0][col] != ".":
            return False
        return trial.drop(col, opponent) is not None and trial.winner(opponent)

```

How it works:

- Win, then block: tries each legal column for AI first, then for opponent. Never blocks if it can win.
- Safe fallback: skips columns where AI piece lets opponent win by stacking above. Prefers center.
- Full columns: only columns where top cell is '.' are considered. Returns None if board is full.
- No board mutation: runs on `copy.deepcopy(board)` so actual board is untouched.

=====

TASK 2 ? COMPLETE GAME TERMINATION & INPUT VALIDATION

=====

```

from board import Board, COLS
from ai import AI
class Game:
    def __init__(self):
        self.board = Board()

```

```
self.ai = AI()
self.turn = "X"
def run(self):
    print("Connect Four -- you are X.")
    while True:
        self.board.print()
        if self.turn == "X":
            try:
                raw = input("Column (1-7), or q: ").strip().lower()
            except (EOFError, KeyboardInterrupt):
                print("\nGoodbye.")
                return
            if raw == "q":
                print("Game quit.")
                return
            try:
                col = int(raw) - 1
            except ValueError:
                print("Please enter a number from 1 to 7, or q to quit.")
                continue
            if not 0 <= col < COLS:
                print("Column must be between 1 and 7.")
                continue
        else:
            col = self.ai.choose_column(self.board)
            if col is None or not 0 <= col < COLS:
                print("AI returned an invalid column; ending game.")
                return
        if self.board.drop(col, self.turn) is None:
            if self.turn == "X":
                print(f"Column {col + 1} is full. Choose another.")
                continue
            print("AI chose a full column; ending game.")
            return
        if self.board.winner(self.turn):
            self.board.print()
            print(self.turn, "wins!")
            return
        if self.board.full():
            self.board.print()
            print("Draw.")
            return
        self.turn = "O" if self.turn == "X" else "X"
```

TASK 4 ? MOVE-LEVEL FEEDBACK

User Prompt:

In game.py, add concise move-level feedback that prints which column the player or AI successfully placed their disc into, appearing exactly once per actual move.

Adding move-announcement messages for player and AI turns:

```
who = "You" if self.turn == "X" else "AI"
print(f"{who} placed a disc in column {col + 1}.")
```

The message prints exactly once per move, for these reasons:

- Only after success: sits after the failed-drop branch. Full-columns and invalid inputs never reach it.
- Before win/draw checks: the winning or drawing move is announced first, followed by final board state.
- Not in loop header: re-prompting on bad input does not duplicate the move message.
- Column numbering: prints col + 1 matching 1-7 numbers entered by user.

Example Output:

You placed a disc in column 4.

AI placed a disc in column 4.

[Claude is AI and can make mistakes.]